

O Apache Kafka como Middleware de Eventos em Sistemas Distribuídos: Arquitetura, Comparativos e Estudo de Caso Prático

Henrique Haag¹
Arthur Wild¹

¹Bacharelado em Ciência da Computação
Universidade do Vale do Rio dos Sinos - UNISINOS
Rio Grande do Sul – RS – Brasil

haaghenrique@edu.unisinos.br

wildarthur@edu.unisinos.br

Abstract. *This paper analyzes Apache Kafka as an event streaming middleware under the perspective of Distributed Systems. We examine Kafka's core architecture, including its commit log structure, partition mechanics, and the KRaft consensus protocol, contrasting it with low-level sockets and traditional queue-based brokers such as RabbitMQ. Furthermore, we explore its practical application in cloud-native core banking architectures, demonstrating its role in transactional reliability, high-throughput replication across datacenters, and modern microservices patterns.*

Resumo. *Este artigo analisa o Apache Kafka como um middleware de streaming de eventos sob a perspectiva de Sistemas Distribuídos. São examinados os fundamentos de sua arquitetura de log de confirmação replicado, mecanismos de partição e o protocolo de consenso KRaft, contrastando-o com sockets de baixo nível e corretores tradicionais de mensageria como o RabbitMQ. Adicionalmente, explora-se sua aplicação prática na modernização de arquiteturas transacionais de core banking nativas da nuvem, evidenciando seu papel na garantia de consistência, replicação geográfica e implementação de padrões modernos de microserviços.*

1. Introdução

Nos sistemas distribuídos modernos, a comunicação eficiente, confiável e robusta entre processos heterogêneos é um dos pilares fundamentais para garantir escalabilidade e tolerância a falhas. Historicamente, a comunicação direta via interfaces de baixo nível, como *sockets* TCP/IP, impunha restrições severas ao design de software corporativo. A comunicação ponto a ponto por *sockets* exige acoplamento temporal (ambos os nós precisam estar online simultaneamente) e acoplamento espacial (os nós precisam conhecer previamente os endereços IP e portas de rede uns dos outros). Além disso, delegar ao desenvolvedor a gestão de buffers de memória, retransmissão de pacotes perdidos e controle de fluxo eleva drasticamente a complexidade do sistema.

Para mitigar tais limitações, surgiram os middlewares de mensageria como camadas de aplicação de software. Embora padrões baseados no modelo de requisição-resposta síncrono (como APIs REST e SOAP sobre HTTP) tenham simplificado a

integração de sistemas, eles introduzem novos gargalos sob alta carga de requisições, induzindo latência acumulada por chamadas sequenciais e aumentando o risco de falhas em cascata no ecossistema de microsserviços [2].

Como resposta a esses desafios, o modelo arquitetural orientado a eventos (Event-Driven Architecture - EDA) consolidou-se ao propor o desacoplamento completo entre as entidades produtoras e consumidoras de dados [2]. Estudos empíricos indicam que a adoção de arquiteturas orientadas a eventos em larga escala pode proporcionar uma redução de até 73,8% na latência de comunicação ponta a ponta e uma melhoria de até 284% no throughput do sistema em comparação com modelos tradicionais de requisição-resposta síncronos [10]. Adicionalmente, verifica-se uma redução média de 76,9% nas dependências diretas de API entre os microsserviços do ecossistema [10]. É nesse paradigma que o **Apache Kafka** se destaca como uma plataforma de streaming de eventos distribuídos de alta disponibilidade, redundância e altíssimo desempenho [1]. Ao contrário de brokers de mensageria que operam com filas efêmeras de consumo destrutivo, o Kafka baseia sua infraestrutura no conceito de um log de commits imutável e replicado, persistido diretamente em disco com algoritmos de acesso em complexidade constante $O(1)$ [1, 6]. Isso permite a múltiplos consumidores independentes ler e reprocessar eventos históricos em seu próprio ritmo (*replayability*) [2, 6].

1.1. Arquiteturas de middleware modernas

As arquiteturas de middleware modernas surgiram para solucionar as limitações de escalabilidade, acoplamento e resiliência das abordagens síncronas tradicionais (como requisições REST/SOAP) e da comunicação de baixo nível por sockets. Em sistemas distribuídos de alta escala, delegar aos serviços a coordenação direta de chamadas síncronas gera gargalos de latência, dependências rígidas e riscos elevados de falhas em cascata. Nesse cenário, as arquiteturas modernas consolidaram a abordagem orientada a eventos (Event-Driven Architecture - EDA) como o paradigma dominante [10]. Estudos apontam que ecossistemas orientados a eventos demonstram uma resiliência significativamente superior durante falhas parciais de infraestrutura, mantendo cerca de 97,65% de disponibilidade de serviços, em contraste com apenas 61,23% de disponibilidade observada em sistemas síncronos equivalentes [10].

O papel principal do middleware moderno é atuar como um barramento de comunicação assíncrono, desacoplando inteiramente os produtores (quem gera a informação) dos consumidores (quem reage a ela) em termos de tempo e espaço. Nesse ínterim, o Apache Kafka se encaixa como a espinha dorsal (*event backbone*) dessas arquiteturas modernas por causa de seus fundamentos de design voltados a alta performance e durabilidade:

- **Log de Commits Replicado:** Ao contrário de middlewares tradicionais que utilizam filas efêmeras de consumo destrutivo, o Kafka implementa o armazenamento de dados baseado em um log de commits distribuído, imutável e persistente em disco. Os eventos não desaparecem após o consumo, o que viabiliza a reprodução (*replayability*) de dados históricos para auditoria, reconstrução de estados de aplicação ou recuperação de falhas.
- **Modelo de Consumo por Pull:** O Kafka utiliza um modelo *pull*, em que o próprio consumidor controla o ritmo de leitura usando um índice sequencial chamado *off-*

set. Isso garante uma gestão eficiente de contrapressão (*backpressure*), impedindo que os consumidores sejam sufocados por picos repentinos de produção.

- **Escalabilidade e Paralelismo via Partições:** A divisão física dos tópicos em partições distribui os logs em paralelo por diferentes servidores (*brokers*). O Kafka atinge altíssimo throughput de processamento mantendo o acesso aos dados em disco em tempo constante $O(1)$, independente de armazenar megabytes ou terabytes de eventos.
- **Consenso Distribuído Integrado:** Alinhado com a busca moderna por infraestruturas de implantação mais simples e com menor pegada operacional, o Kafka evoluiu para eliminar a dependência externa do ZooKeeper em favor do KRaft (Kafka Raft), embutindo o algoritmo de consenso Raft nativamente dentro de seu próprio quórum de controladores.

Enquanto middlewares síncronos de alta tipagem como o gRPC atuam de forma próxima a sockets tradicionais com contratos fortemente acoplados, e middlewares de mensageria como o RabbitMQ primam pelo gerenciamento inteligente e granular de itens de trabalho efêmeros, o Kafka se destaca nas arquiteturas modernas como uma infraestrutura estratégica para streaming massivo de dados em tempo real.

2. O Apache Kafka: Arquitetura e Funcionamento

O Apache Kafka foi concebido originalmente no LinkedIn para rastrear comportamentos de usuários em tempo real e, posteriormente, doado à *Apache Software Foundation* [1]. Ele se consolidou como um barramento de mensageria distribuído, escalável, redundante e de alto desempenho [1]. Esta seção descreve a organização interna do middleware, seus mecanismos de consumo e sua evolução em termos de consenso distribuído sob a perspectiva de Sistemas Distribuídos.

2.1. Modelo de Log Replicado e Estrutura Física

Diferente de sistemas de mensageria tradicionais que utilizam filas efêmeras de consumo destrutivo, o Kafka implementa o armazenamento de dados com base em um *commit log* distribuído e replicado [1]. Os dados são estruturados em tópicos, os quais são divididos logicamente em partições [1]. Cada partição funciona como um arquivo de log ordenado e imutável do tipo *append-only*, onde cada nova mensagem é anexada sequencialmente ao final da estrutura [1]. Internamente, os logs de cada partição são fragmentados em arquivos físicos denominados segmentos de log (*log segments*), gerenciados por limites de tamanho (geralmente 1 GB) ou de tempo [1].

A distribuição física dessas partições ocorre entre os servidores do cluster, chamados de *brokers* [1]. Para garantir tolerância a falhas e alta disponibilidade, cada partição possui um fator de replicação associado [1]. Dentre as réplicas de uma partição, uma é eleita como líder (*leader*), enquanto as demais atuam como seguidoras (*followers*) [1]. Toda operação de escrita e leitura realizada pelos clientes (produtores e consumidores) é direcionada para a réplica líder [1]. As réplicas seguidoras limitam-se a replicar de forma assíncrona os registros da líder para fins de redundância [1].

As réplicas seguidoras que conseguem acompanhar o ritmo de gravação do líder são classificadas como réplicas em sincronia (*In-Sync Replicas - ISR*) [1]. O nível de durabilidade das escritas é controlado pela configuração `acks` no produtor:

- `acks=0`: O produtor não aguarda confirmação do broker, enviando os dados sem garantia de recebimento para maximizar a vazão de rede [1].
- `acks=1`: O líder envia a confirmação de sucesso imediatamente após anexar a mensagem ao seu log local de escrita, sem esperar a replicação das seguidoras [1].
- `acks=all`: O líder aguarda que todas as réplicas ativas dentro do conjunto ISR repliquem e confirmem a escrita localmente antes de enviar a confirmação de sucesso ao produtor, garantindo resiliência total contra perda de dados em caso de falha de infraestrutura [1].

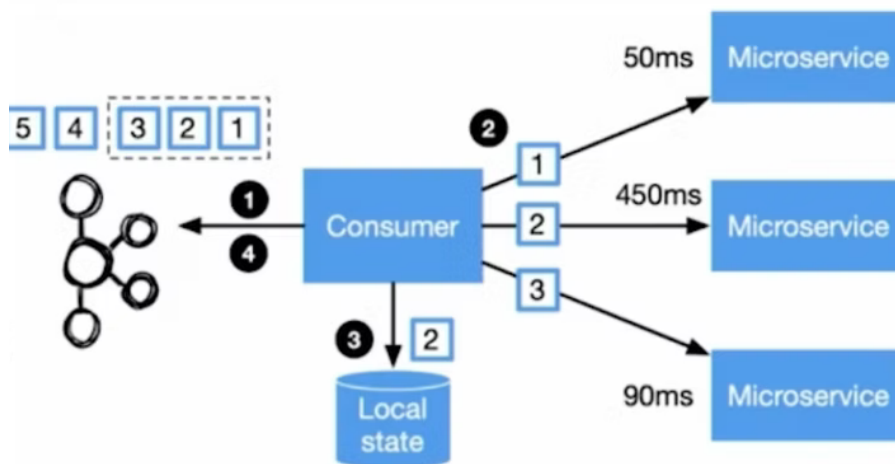


Figure 1. Arquitetura de distribuição de partições do Kafka. Fonte: [4].

2.2. Semântica de Consumo, Offset e Paralelismo

No Kafka, a leitura de dados ocorre de maneira não destrutiva: as mensagens permanecem persistidas em disco físico de acordo com as políticas de retenção configuradas, permitindo leituras repetidas por diferentes aplicações [1]. O controle da leitura é descentralizado, sendo de responsabilidade do consumidor através do conceito de *offset* — um índice numérico sequencial imutável que indica a posição de leitura atualizada dentro da partição de um tópico [1].

Para viabilizar escalabilidade horizontal, o Kafka organiza os consumidores em grupos de consumo (*consumer groups*) [1]. Dentro de um mesmo grupo, cada partição de um tópico é atribuída de forma exclusiva a, no máximo, um único consumidor ativo, o que evita duplicidade no processamento das mensagens [1].

Para mitigar problemas com registros corrompidos ou falhas persistentes no processamento (as chamadas mensagens venenosas ou *poison messages*), as arquiteturas de microserviços modernas comumente implementam políticas de retentativa com *backoff* e o posterior encaminhamento automático desses registros para tópicos de mensagens mortas (*Dead-Letter Queues* — DLQ) para análise assíncrona isolada, assegurando que o grupo de consumidores não seja paralisado indefinidamente [11].

Para dimensionar corretamente o número de partições (P) em produção com base na vazão total desejada do sistema (TT), aplica-se a seguinte formulação teórica [1]:

$$P = \max(N_P, N_C) \quad (1)$$

Onde N_P representa o número de produtores necessários, determinado pela divisão da vazão esperada do sistema pela capacidade máxima de um único produtor ($N_P = TT/TP$) [1]. Da mesma forma, N_C define o número de consumidores necessários para processar o fluxo sem atrasos, baseado no throughput máximo de consumo por partição ($N_C = TT/TC$) [1].

2.3. Consenso e Metadados do Cluster: Do ZooKeeper ao KRaft

Historicamente, a coordenação distribuída dos metadados do Kafka (como estados dos brokers, tópicos e líderes de partições) dependia de um cluster externo do Apache ZooKeeper [1]. Contudo, essa arquitetura de duas camadas gerava sobrecarga de rede, atrasos na sincronização de metadados em larga escala e risco de falhas de consistência durante eleições de novos líderes [1].

Com a introdução do protocolo KRaft (*Kafka Raft*), o gerenciamento de metadados passou a ser embutido nativamente no cluster através de um quórum de controladores (*controllers*) baseados em Raft [1]. Nesse modo, um set de brokers especializados, chamados de controladores, é implantado para formar um cluster focado na manutenção do estado de metadados [1]. Um controlador ativo é eleito como líder do quórum de controladores para centralizar as alterações de estado, gravando-as no tópico interno de partição única `_cluster_metadata` [1]. Os outros controladores replicam ativamente esse tópico (atuando no conjunto ISR), enquanto os brokers tradicionais de dados operam apenas como observadores (*observers*), mantendo um cache local de metadados assíncrono que é atualizado periodicamente a partir das requisições de consulta (*heartbeats*) enviadas ao líder [1].

A consistência dos metadados é mantida pela definição do *High Watermark* (HWM) adaptada no KRaft: diferentemente da replicação de dados comum do Kafka, o HWM do KRaft representa o *offset* mais alto que foi replicado pela maioria absoluta do quórum de controladores, seguindo rigorosamente o modelo matemático do Raft [1]. No caso de falhas e eleição de um novo líder de metadados, o quórum vota de forma baseada em timeout utilizando o maior log ou época de cada candidato [1]. Uma vez eleito, as réplicas seguidoras executam a reconciliação de log (*log reconciliation*), truncando seus registros de metadados divergentes até a posição demarcada pelo HWM para depois sincronizar os novos eventos ditados pelo líder [1].

3. Comparação Técnica: Apache Kafka vs. RabbitMQ

A comparação entre middlewares de mensageria assíncrona exige contrastar seus paradigmas fundamentais de armazenamento, consistência e roteamento. O RabbitMQ e o Apache Kafka nasceram em extremos opostos do problema e convergiram ao longo do tempo (KIP-932 no Kafka [8] e Streams no RabbitMQ [8]). A Tabela ?? sintetiza as diferenças técnicas estruturais.

	Apache Kafka	RabbitMQ
Paradigma Principal	Log de commits distribuído imutável.	Mensageria inteligente e flexível baseada em Exchanges, Bindings e Filas.
Estrutura Física	Tópico dividido em Partições distribuídas e replicadas em disco.	Filas tradicionais temporárias/destrutivas (Classic/Quorum Queues) ou Logs append-only (Streams).
Unidade da Mensagem	Posição sequencial imutável (offset) dentro de um log compartilhado.	Item de trabalho independente com metadados e ciclo de vida próprios no envelope.
Roteamento de Dados	No cliente (client-side): o produtor decide para qual partição enviar (via hash de chave); o broker é agnóstico ao conteúdo.	No servidor (broker-side): Exchanges aplicam regras dinâmicas (Bindings) para direcionar cópias às filas.
Garantia de Durabilidade	Baseia-se primariamente em replicação ativa (ISR). Desabilita o fsync síncrono por padrão, confiando no page cache.	Quorum Queues: replicadas via Raft, exigem gravação e fsync físico da maioria antes do reconhecimento (publisher confirm).
Semântica de Consumo	Tradicionalmente por partições exclusivas via Consumer Groups. A partir de 2026 (v4.2), aceita Share Groups (consumidores concorrentes).	Consumidores concorrentes que competem pela carga de trabalho de uma mesma fila.
Filtragem de Dados	No cliente (client-side): o consumidor precisa baixar todos os registros da partição para filtrá-los localmente.	No servidor (server-side): filtros de Bloom e expressões SQL limitam o tráfego de rede enviando apenas o dado requisitado.
Controle Fino de Mensagens	Baseado na política de retenção global do tópico (tempo ou espaço em disco).	Toolkit individual: atraso de entrega, retentativa com backoff dinâmico, prioridade estrita (32 níveis), TTL e dead-lettering.

	Apache Kafka	RabbitMQ
Protocolos Suportados	Protocolo binário proprietário via TCP.	Multi-protocolo nativo: AMQP 0-9-1, AMQP 1.0, MQTT, STOMP e protocolo de Stream próprio.
Consenso de Metadados	Quorum de controladores unificado e interno via KRaft (eliminando a necessidade de ZooKeeper externo).	Consenso e estado interno do cluster gerenciado via Khepri (consenso baseado em Raft).

Além do detalhamento exposto na tabela, a principal distinção operacional reside no acoplamento de roteamento. Enquanto o RabbitMQ permite aos clientes redefinir topologias de comunicação de forma dinâmica em tempo de execução via *bindings* e *exchanges* [8], o Kafka impõe a rigidez do particionamento definido no momento da gravação pelo cliente produtor [1]. Adicionalmente, as garantias de durabilidade divergem significativamente: o compromisso do Kafka reside em replicação em memória ativa (page cache) sem garantia física de escrita síncrona em disco (*fsync*) no caminho crítico de gravação, o que pode incorrer em perda de dados em cenários de queda catastrófica simultânea de energia de todas as réplicas ISR [8]. As *Quorum Queues* do RabbitMQ, por outro lado, garantem conformidade transacional total realizando *fsync* síncrono consolidado por lote via WAL compartilhado entre múltiplos quóruns Raft antes de emitir a confirmação de escrita (*publisher confirm*) [8].

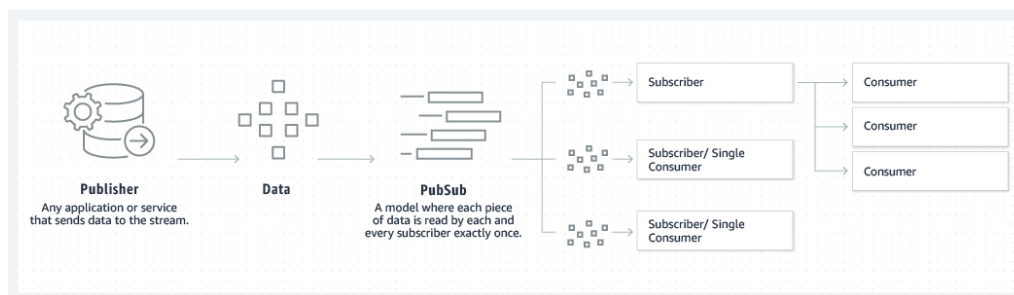


Figure 2. Arquitetura de publicação-assinatura tradicional. Fonte: [6].

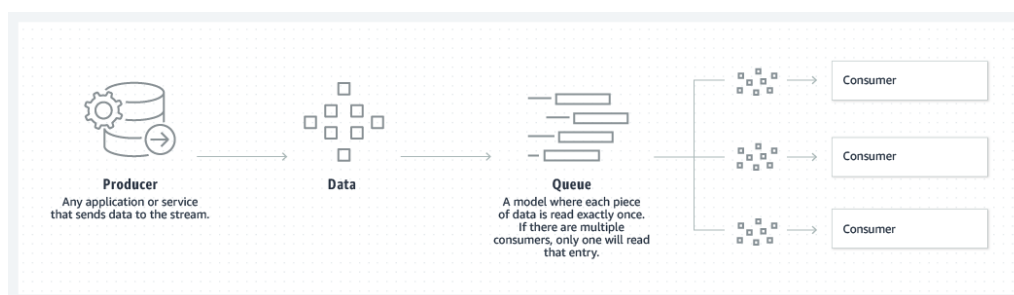


Figure 3. Funcionamento de canais baseados em filas. Fonte: [6].

4. Estudo de Caso Prático: Modernização de Core Banking Nativo da Nuvem

Os sistemas de *core banking* representam o barramento transacional de missão crítica de instituições financeiras, englobando desde transações de depósitos e saques até fluxos de conformidade e KYC (*Know Your Customer*) [4]. Historicamente operados sobre *mainframes* legados com base em Cobol e IBM DB2, as empresas buscam modernizar essas infraestruturas por razões de custo, agilidade de mercado e restrições de escalabilidade física [4].

4.1. Cargas de Trabalho Transacionais vs. Analíticas e Requisitos de Resiliência

Em uma plataforma bancária moderna, as cargas de trabalho transacionais e analíticas possuem características de desempenho e acordos de nível de serviço (SLA) fundamentalmente distintos [4]. Atividades analíticas toleram janelas de indisponibilidade e metas de RPO maiores do que zero [4]. Já as cargas transacionais exigem RPO estritamente igual a zero e RTO o mais próximo possível de zero [4].

Para atender a esse SLA em produção, o Apache Kafka é implantado em arquiteturas altamente resilientes conhecidas como *Stretch Clusters* [1]. Essa abordagem distribui de forma síncrona um único cluster lógico do Kafka entre datacenters geográficos distintos ou zonas de disponibilidade com latência máxima recomendada de 50 ms [1]. O mecanismo utiliza a configuração de *rack awareness* através do parâmetro `broker.rack` para instruir o Kafka a distribuir as partições fisicamente entre diferentes datacenters [1]. Produtores operando sobre transações críticas utilizam obrigatoriamente `acks=all`, garantindo que a escrita só seja validada após a replicação síncrona no page cache de todas as réplicas que compõem o ISR [1, 8]. Em conjunto com a definição rigorosa de `min.insync.replicas`, assegura-se matematicamente que ao menos um datacenter independente gravou a informação de forma íntegra antes da conclusão da transação [1].

4.2. Modelos de Sucesso em Core Banking Moderno

Três grandes implementações demonstram o impacto prático dessa arquitetura:

- **Thought Machine (Vault):** Sistema operacional de core banking nativo de nuvem operando transações 24/7 com perda zero de dados [4]. O Vault utiliza o processamento de eventos do Kafka em seu coração para habilitar processamento assíncrono e paralelização em escala, estabelecendo padrões para garantir integridade operacional nos fluxos paralelos [4].
- **10x Banking:** Plataforma baseada em design orientado a domínios que adota o Kafka como seu barramento central de dados analíticos e transacionais [4]. O 10x Banking emprega a arquitetura Kappa combinada com Apache Pinot e Trino para consolidar fluxos em tempo real e em lote em um único pipeline de dados, reduzindo drasticamente o tempo de mercado para novos produtos bancários personalizados [4].
- **Custodigit:** Plataforma voltada a investimentos de ativos digitais e criptomoedas em ambiente severamente regulado [4]. Utiliza o Kafka como sistema nervoso central de sua arquitetura de microsserviços, coordenando a coreografia de processos com estado por meio de microsserviços desenvolvidos com a API do *Kafka Streams* e aplicando o padrão transacional distribuído *Saga* sob semântica exatamente-uma-vez [4].

O redirecionamento de consultas e escritas de mainframes legados para o Apache Kafka (*mainframe offloading*) proporciona economias significativas de custos operacionais. O Royal Bank of Canada (RBC) economizou milhões de dólares em despesas fixas (OPEX) ao descarregar logs de transações de mainframe para o Kafka, viabilizando novos aplicativos orientados a eventos sem onerar as máquinas legadas centrais [4].

5. Conclusão

A análise do Apache Kafka sob a ótica de Sistemas Distribuídos consolida a transição dos modelos de comunicação síncronos ponto a ponto (sockets) para arquiteturas orientadas a eventos em larga escala [1, 2]. Ao abstrair a comunicação direta de transporte por sockets em favor de um log distribuído e persistente com complexidade $O(1)$ [1], o Kafka remove os acoplamentos espaciais e temporais do caminho crítico de execução das aplicações [2, 6].

Nesse sentido, a consolidação dessas plataformas nos cenários corporativo e acadêmico evidencia a relevância do estudo do Kafka em disciplinas de Redes de Computadores e Sistemas Distribuídos, servindo como uma infraestrutura estratégica indispensável para o desenvolvimento de soluções robustas alinhadas com as demandas de alta escala de empresas como Netflix e Uber [12].

Contudo, a escolha do middleware exige avaliar compensações técnicas fundamentais. O Kafka delega maior responsabilidade de roteamento e consistência às aplicações clientes [1, 8]. A sua resiliência transacional fundamenta-se estritamente na redundância do cluster e na eficiência da replicação ativa via page cache das ISRs, sem persistência física síncrona (*fsync*) individualizada de mensagens [8].

Dessa forma, o Apache Kafka consolida-se como o padrão de engenharia ideal quando os requisitos do sistema demandam processamento contínuo de dados de alto throughput, semântica exatamente-uma-vez, replay de logs históricos e processamento stateful em tempo real [1, 4]. Se o cenário demandar roteamento inteligente de mensagens no servidor, latência sob controle individual fino de fila ou durabilidade de mensagem persistida sob flush físico imediato no disco de cada nó, middlewares com design de fila tradicional, tais como o RabbitMQ, permanecem recomendados [8].

References

- [1] Cloudera Inc. (2026). Apache Kafka Overview. *Cloudera Runtime 7.3.2 Documentation*.
- [2] Holigi, C. (2025). Building Event-Driven Microservices with Apache Kafka: A Practical Architecture for High-Scale Platforms. *DEV Community*.
- [3] IT Solutionss (2024). Curso Apache Kafka com Kubernetes e Docker. *IT Solutionss Treinamentos*.
- [4] Busko, P. (2023). Modernização do Core Banking nativo da nuvem com Apache Kafka. *Blog Pedro Busko - Streaming de Dados*.
- [5] Tebaldi, P. (2026). Monitoramento de Kafka: métricas, ferramentas e alertas. *OpServices Blog*.
- [6] Amazon Web Services (2026). O que é o Kafka? — Explicação sobre o Apache Kafka. *AWS Cloud Concepts*.

- [7] Apache Software Foundation (2026). Overview — Apache Kafka. *Apache Kafka Official Documentation*.
- [8] Broadcom Inc. (2026). RabbitMQ vs. Apache Kafka. *RabbitMQ Official Documentation*.
- [9] Cetax (2026). Tendências da Engenharia de Dados para 2026: IA, Lakehouse, Data Mesh e o Futuro dos Dados. *Cetax Blog*.
- [10] Thondalapally, K. R. (2025). Event-Driven Architectures: The Foundation of Modern Distributed Systems. *International Journal on Science and Technology (IJSAT)*, 16(1).
- [11] Koder.ai (2026). O que é o Kafka e como ele é usado em sistemas modernos? *Koder.ai Blog*.
- [12] Gomes, G. e Júnior, J. (2025). Trabalho Final - Apache Kafka. *EEL878 - Redes de Computadores I, Universidade Federal do Rio de Janeiro (UFRJ)*.